APPROACH

GitHub Copilot - AI Augmented Quality Engineer

https://myskillspring.cognizant.com/learner/assemblyDetails/a72d42d9-5cdf-473f-8b99-a1b43893130c

Smart university
Parabank

Start assessment > Pre-requisite course > Tekstac > VDI > VS code > 500733_cgcp > Company mail ID > Authetication

500733_cgcp

Use Claude sonnet 5

Give it as single prompt

It takes 30 mins to run (should give allow)

Defect report generated

Parabank Question

Description

UI AUTOMATION — PARABANK REGISTRATION, OPEN NEW ACCOUNT & FUND TRANSFER

Supports: Any automation tool (Selenium,Playwright,Cypress) in any language (Java,JavaScript, TypeScript, Python, C#, .NET)

OVERVIEW

This assignment requires you to analyze requirements, generate test cases, automate scenarios, execute tests, identify defects, and submit a structured defect report for the Registration, Open New Account, and Fund Transfer modules of the ParaBank application.

IMPORTANT: HOW THIS ASSIGNMENT IS EVALUATED

This assignment does NOT just evaluate the quality of your final code or test cases. It evaluates HOW you collaborated with AI to produce them.

The evaluation measures two things separately:

1. What you analyzed and discovered — and whether you shared it with AI (Application Analysis, Acceptance Criteria, Validation Rules)
2. How you structured and directed your AI prompts (Research prompts, generation prompts, code prompts, refinement prompts)

IMPORTANT GUIDELINES

Do not change the provided workspace structure at any cost. You may implement your framework on top of the existing workspace, but the structure must remain unchanged.

Execution Method: Use the integrated terminal in VS Code for all test execution.

Avoid redundant files and folders and keep the workspace clean and organized.

Create a single defect report named defect_report.json consolidating all identified defects. This file must be present in your workspace after execution.

Follow industry-standard best practices for framework design, naming conventions, and code organization.

APPLICATION UNDER TEST

Registration: http://webapps.tekstac.com:9090/tekbank/register.htm

The application URL has also been provided in the **user stories** available in your project workspace.

Open New Account and Fund Transfer: Access these modules by navigating the application after login. No separate entry URLs are required.

SCOPE

Module 1 — Registration: 11 fields, US-01 to US-07
Module 2 — Open New Account: Account Type and Source Account, US-08 to US-10
Module 3 — Fund Transfer: Amount and Account Selection, US-11 to US-13

Note: Please refer to the use cases provided in the user story, as outlined in the project template.

HOW TO APPROACH THIS EXERCISE

Each step below corresponds to criteria that are scored in your evaluation.
Skipping a step means scoring 0 on the corresponding criteria.

Step 1 — Inspect All Three Modules
Open each form. Identify HTML element IDs, field types, and behaviors.
Submit empty forms and invalid values to observe how the application responds.

Step 2 — Define Acceptance Criteria
Create acceptance criteria for user stories US-01 through US-13.
AI assistance is allowed, but your prompts must show you are driving the definition.
Save your findings in **Application Analysis & Requirements.txt.**

Step 3 — Research Validation Rules
Identify and document all field-level validation rules for all three modules.
Add these rules to Application Analysis & Requirements.txt before proceeding.

Step 4 — Generate Comprehensive Test Cases With Full Context (.csv)
Use the acceptance criteria, element IDs, and validation rules gathered in Steps 1 to 3 as context in your prompt. Include all user story links and specify the required test distribution before generating.

Step 5 — Generate Automation Code
Use AI to implement browser setup, page objects, test methods, and helper utilities.
Reference the element IDs and validation rules from your analysis in each prompt.

Step 6 — Refine Outputs
Run the generated tests and submit follow-up prompts to fix failures, correct assertions, and improve coverage based on what you observe during execution.

Step 7 — Document Defects
Execute tests and document defects based on actual observed behavior.
Each defect must include what was entered, what happened, and what was expected.
Save all defects in a single file named **defect_report.json** in your workspace.

VALIDATION RULES

Registration Module

Name fields and City	Letters only — numbers and special characters must be rejected
State	Exactly 2 alphabetic characters
ZIP Code	Exactly 5 numeric digits
Phone	Exactly 10 numeric digits
SSN	Exactly 9 numeric digits
All 11 fields	Mandatory — empty submission must show a validation error

Open New Account Module

Account Type	Must be selected (CHECKING or SAVINGS)
Source Account	Must be selected from dynamically loaded dropdown
$100.00 label	This is a static display label — not a field to test or interact with

Fund Transfer Module

Amount	Mandatory, must be numeric, must be positive (greater than zero)
From Account	Must be selected from dynamically loaded dropdown
To Account	Must be selected from dynamically loaded dropdown

TASKS

Task 1 — Test Case Generation
Generate at least 60 detailed test cases in CSV format covering all 13 user stories across all 3 modules.

Required Columns:
TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Priority Values: Critical, High, Medium, Low

Distribution: 60-70% of cases must be negative/validation tests (Critical or High priority). Positive/success scenarios should be Low priority. Boundary tests should be Medium.

Task 2 — Automation Implementation
Create a complete automation solution covering all three modules.
Implement at least 20 automated test methods with assertions.
Prioritize critical and high-risk validations over happy-path scenarios.

Task 3 — Execution and Bug Detection
Execute automation. Provide execution evidence. Save all defects in a single consolidated file named defect_report.json. Detect and document at least 8 validation defects across all modules. Defects must be from actual test execution — not AI-predicted.

DELIVERABLES

1. Application Analysis & Requirements.txt — your documented analysis of all three modules including element IDs, acceptance criteria, and validation rules
2. Test case document — 60+ test cases covering all three modules in a csv file format
3. Automation codebase — page objects for Registration, Open New Account, Fund Transfer
4. defect_report.json — one consolidated defect report in JSON format capturing all defects from actual test execution

MINIMUM PASSING EXPECTATIONS

Application Analysis & Requirements.txt present in workspace with documented findings
60+ test cases covering all 13 user stories
Risk-based coverage with 60-70% negative test scenarios
At least 20 automated tests with meaningful assertions
Page objects for Registration, Open Account, and Transfer pages

defect_report.json present in workspace with at least 8 documented defects,
each including expected result, actual result, severity, and priority

==========

APPLICATION ANALYSIS & REQUIREMENTS DOCUMENT

Target Application: ParaBank Demo (Tekstac Hosted)

Base Registration URL: http://webapps.tekstac.com:9090/tekbank/register.htm

==========

1. MODULE & FIELD ELEMENT ID MAP

Module 1: Registration Page (http://webapps.tekstac.com:9090/tekbank/register.htm)

- First Name : id="customer.firstName"
- Last Name : id="customer.lastName"
- Address (Street) : id="customer.address.street"
- City : id="customer.address.city"
- State : id="customer.address.state"
- Zip Code : id="customer.address.zipCode"
- Phone Number : id="customer.phoneNumber"
- SSN : id="customer.ssn"
- Username : id="customer.username"
- Password : id="customer.password"
- Confirm Password : id="repeatedPassword"
- Register Button : input[type="submit"][value="Register"]

Module 2: Open New Account Page (Post-login Navigation)

- Account Type : select id="type" (Options: CHECKING, SAVINGS)
- Source Account : select id="fromAccountId" (Dynamic Loading Dropdown)
- $100.00 Label : Static text element (Not interactive)
- Open Account Button : input[type="submit"][value="Open New Account"]

Module 3: Fund Transfer Page (Post-login Navigation)

- Transfer Amount : id="amount"
- From Account : select id="fromAccountId" (Dynamic Loading Dropdown)
- To Account : select id="toAccountId" (Dynamic Loading Dropdown)
- Transfer Button : input[type="submit"][value="Transfer"]

2. VALIDATION RULES MATRIX

Field / Module	Rule Description & Constraints
First / Last / City	Letters only. Reject numbers and special characters.
State	Exactly 2 alphabetic characters (e.g., "CA", "NY").
Zip Code	Exactly 5 numeric digits (e.g., "90210").
Phone Number	Exactly 10 numeric digits (e.g., "9876543210").
SSN	Exactly 9 numeric digits (e.g., "123456789").
All 11 Reg Fields	Mandatory. Blank submissions must trigger validation errors.
Account Type	Mandatory selection (CHECKING or SAVINGS).
Source/From Account	Mandatory selection from dynamically loaded options.
To Account	Mandatory selection from dynamically loaded options.
Transfer Amount	Mandatory, numeric, positive value (> 0.00).

3. ACCEPTANCE CRITERIA (US-01 through US-13)

US-01 [Reg - Names]: First Name and Last Name accept alphabetic characters only; show errors for digits/symbols.

US-02 [Reg - Address]: Street accepts standard address format; City accepts letters only.

US-03 [Reg - State]: State accepts exactly 2 uppercase/lowercase alphabetic characters.

US-04 [Reg - Zip]: Zip Code accepts exactly 5 numeric digits.

US-05 [Reg - Phone & SSN]: Phone requires 10 numeric digits; SSN requires 9 numeric digits.

US-06 [Reg - Credentials]: Username, Password, and Confirm Password must match and meet security length rules.

US-07 [Reg - Mandatory]: Form submission without required fields blocks submission with clear field errors.

US-08 [Open Acc - Type]: User can select valid Account Type (CHECKING/SAVINGS).

US-09 [Open Acc - Source]: Source Account dropdown populates dynamically with existing user accounts.

US-10 [Open Acc - Submission]: Submitting valid account parameters opens new account and shows confirmation ID.

US-11 [Transfer - Amount]: Amount field accepts positive numeric values (>0); blocks negative/zero/non-numeric.

US-12 [Transfer - Selection]: From Account and To Account dropdowns populate dynamically with valid accounts.

US-13 [Transfer - Execution]: Valid transfer updates account balances and displays transaction success page.

Userstories txt:

SCOPE

Module 1 — Registration: 11 fields, US-01 to US-07

Module 2 — Open New Account: Account Type and Source Account, US-08 to US-10

Module 3 — Fund Transfer: Amount and Account Selection, US-11 to US-13

Note: Please refer to the use cases provided in the user story, as outlined in the project template.

HOW TO APPROACH THIS EXERCISE

Each step below corresponds to criteria that are scored in your evaluation.

Skipping a step means scoring 0 on the corresponding criteria.

Step 1 — Inspect All Three Modules

Open each form. Identify HTML element IDs, field types, and behaviors.

Submit empty forms and invalid values to observe how the application responds.

Step 2 — Define Acceptance Criteria

Create acceptance criteria for user stories US-01 through US-13.

AI assistance is allowed, but your prompts must show you are driving the definition.

Save your findings in Application Analysis & Requirements.txt.

Step 3 — Research Validation Rules

Identify and document all field-level validation rules for all three modules.

Add these rules to Application Analysis & Requirements.txt before proceeding.

Step 4 — Generate Comprehensive Test Cases With Full Context (.csv)

Use the acceptance criteria, element IDs, and validation rules gathered in Steps 1 to 3 as context in your prompt. Include all user story links and specify the required test distribution before generating.

Step 5 — Generate Automation Code

Use AI to implement browser setup, page objects, test methods, and helper utilities.

Reference the element IDs and validation rules from your analysis in each prompt.

Step 6 — Refine Outputs

Run the generated tests and submit follow-up prompts to fix failures, correct assertions, and improve coverage based on what you observe during execution.

Step 7 — Document Defects

Execute tests and document defects based on actual observed behavior.

Each defect must include what was entered, what happened, and what was expected.

Save all defects in a single file named defect_report.json in your workspace.

VALIDATION RULES

Registration Module

Name fields and City	Letters only — numbers and special characters must be rejected
State	Exactly 2 alphabetic characters
ZIP Code	Exactly 5 numeric digits
Phone	Exactly 10 numeric digits
SSN	Exactly 9 numeric digits
All 11 fields	Mandatory — empty submission must show a validation error

Open New Account Module

Account Type	Must be selected (CHECKING or SAVINGS)
Source Account	Must be selected from dynamically loaded dropdown
$100.00 label	This is a static display label — not a field to test or interact with

Fund Transfer Module

Amount	Mandatory, must be numeric, must be positive (greater than zero)
From Account	Must be selected from dynamically loaded dropdown
To Account	Must be selected from dynamically loaded dropdown

TASKS

Task 1 — Test Case Generation

Generate at least 60 detailed test cases in CSV format covering all 13 user stories across all 3 modules.

Required Columns:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Priority Values: Critical, High, Medium, Low

Distribution: 60-70% of cases must be negative/validation tests (Critical or High priority).

Positive/success scenarios should be Low priority. Boundary tests should be Medium.

Task 2 — Automation Implementation

Create a complete automation solution covering all three modules.

Implement at least 20 automated test methods with assertions.

Prioritize critical and high-risk validations over happy-path scenarios.

Task 3 — Execution and Bug Detection

Execute automation. Provide execution evidence. Save all defects in a single consolidated file named defect_report.json. Detect and document at least 8 validation defects across all modules. Defects must be from actual test execution — not AI-predicted.

DELIVERABLES

1. Application Analysis & Requirements.txt — your documented analysis of all three modules including element IDs, acceptance criteria, and validation rules

2. Test case document — 60+ test cases covering all three modules in a csv file format

3. Automation codebase — page objects for Registration, Open New Account, Fund Transfer

4. defect_report.json — one consolidated defect report in JSON format capturing all defects from actual test execution

Test evalution:
URL: https://github.com/enterprises/CognizantGitHubCopilot

Parabank Answer

✅ MASTER PROMPT — PARA BANK UI AUTOMATION (SHORT & EFFECTIVE)

You are a **Senior QA Automation Engineer + SDET Expert**.

Your task is to complete a **UI Automation Assessment for ParaBank Application** covering:

- Registration Module (US-01 to US-07)
- Open New Account Module (US-08 to US-10)
- Fund Transfer Module (US-11 to US-13)

Use **industry best practices** in:

- Test design
- Automation framework design
- Validation handling
- Defect reporting

You MUST follow ALL instructions strictly.

🔹 APPLICATION DETAILS

Registration URL:
http://webapps.tekstac.com:9090/tekbank/register.htm

Other modules are accessible after login.

🔹 ELEMENT IDs (USE EXACTLY)

Registration

- customer.firstName
- customer.lastName

- customer.address.street
- customer.address.city
- customer.address.state
- customer.address.zipCode
- customer.phoneNumber
- customer.ssn
- customer.username
- customer.password
- repeatedPassword

Open New Account

- type
- fromAccountId

Fund Transfer

- amount
- fromAccountId
- toAccountId

🔹 VALIDATION RULES (STRICT)

Registration

- Names + City → letters only
- State → exactly 2 letters
- ZIP → exactly 5 digits
- Phone → exactly 10 digits
- SSN → exactly 9 digits
- All 11 fields mandatory

Open Account

- Account Type → required
- Source Account → required (dynamic dropdown)
- Ignore "$100.00" label

Fund Transfer

- Amount → required, numeric, > 0
- From Account → required
- To Account → required

🔷 ACCEPTANCE CRITERIA (US-01 to US-13)

Use these EXACTLY while generating outputs:

- US-01 → Name validation
- US-02 → Address & City validation
- US-03 → State validation
- US-04 → ZIP validation
- US-05 → Phone & SSN validation
- US-06 → Username/Password match
- US-07 → Mandatory fields validation
- US-08 → Account type selection
- US-09 → Source account population
- US-10 → Account creation success
- US-11 → Transfer amount validation
- US-12 → Account dropdown population
- US-13 → Successful fund transfer

🔷 STEP-BY-STEP EXECUTION (DO NOT SKIP)

Step 1 — Application Analysis

- Map all elements
- Identify field types
- Observe behavior for invalid/empty inputs

Step 2 — Acceptance Criteria

- Define for US-01 to US-13

Step 3 — Validation Rules

- Document all field-level validations

👉 Save Step 1–3 output in:
Application Analysis & Requirements.txt

Step 4 — Test Case Generation (CSV)

Generate **≥ 60 test cases** with:

Columns:
TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Rules:

- 60–70% → Negative tests (Critical/High)
- Positive tests → Low
- Boundary tests → Medium
- Cover ALL 13 user stories

Step 5 — Automation Code

Generate a **complete automation framework** using ANY ONE:
(Selenium / Playwright / Cypress)

Include:

- Base setup
- Page Object Model (POM)
- Utility classes
- Test classes

Must implement:

- ≥ 20 automated test methods
- Assertions for validations
- Use given element IDs

Step 6 — Refinement

- Improve failing tests
- Fix locators/assertions
- Increase validation coverage

Step 7 — Defect Reporting

After execution:

Create **defect_report.json** with:

- At least 8 REAL defects
- Each defect must include:
    - Module
    - Steps
    - Test Data
    - Expected Result
    - Actual Result
    - Severity
    - Priority

🔷 DELIVERABLES (MANDATORY)

Application Analysis & Requirements.txt

- Must include:
    - Element IDs
    - Acceptance Criteria (US-01 to US-13)
    - Validation Rules

TestCases.csv

- Minimum 60 test cases
- Must include all required columns:
    - TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Automation Codebase

- Page Object Model (POM)
- Separate pages for:
    - Registration
    - Open New Account
    - Fund Transfer
- ≥ 20 test methods with assertions

defect_report.json

- Single consolidated file
- Minimum 8 defects
- Each defect must include:
    - Module
    - Steps

    - Test Data
    - Expected Result
    - Actual Result
    - Severity
    - Priority

🔹 IMPORTANT RULES

- DO NOT change workspace structure
- Keep project clean (no redundant files)
- Use VS Code terminal for execution
- Follow naming conventions
- Prioritize validation-heavy testing

🔹 OUTPUT FORMAT REQUIREMENTS

- Test cases → CSV format
- Code → structured files
- Defects → JSON
- Analysis → plain text

✅ FINAL INSTRUCTION

Generate outputs **step-by-step (NOT all at once)**.
Wait for user confirmation before moving to next step.

If you want, I can also give you **step-wise mini prompts (Step 1 → Step 7)** like last time so you can *drive the AI perfectly and score higher*.

Smart University Question

Time left
01 : 59
hrs min
Submit

1

Description

UI AUTOMATION — SMARTUNIVERSITY: LOGIN, ADD STUDENT & ADD PROFESSOR

Supports: Any automation tool (Cypress, Playwright, Selenium) in any language
(JavaScript, TypeScript, Python, Java, C#, .NET)

OVERVIEW

This assignment requires you to analyze requirements, generate test cases, automate
scenarios, execute tests, identify defects, and submit a structured defect report for
the Login, Add student, and Add Professor modules of the Smart University application.

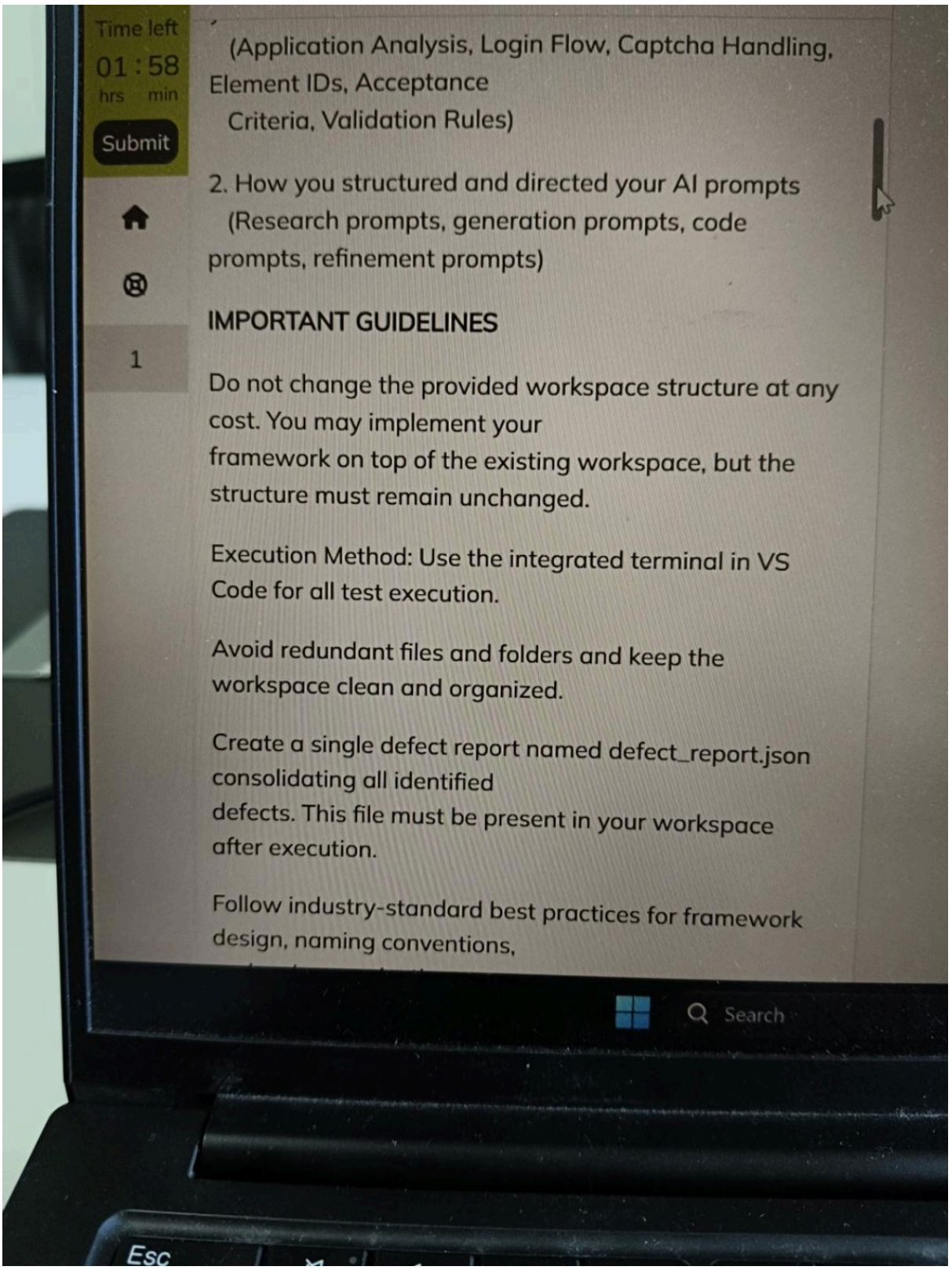
IMPORTANT: HOW THIS ASSIGNMENT IS EVALUATED

This assignment does NOT just evaluate the quality of your final code or test cases.
It evaluates HOW you collaborated with AI to produce them.

The evaluation measures two things separately:

1. What you analyzed and discovered — and whether you shared it with AI

Search


Time left
01 : 58
hrs min
Submit
1
(Application Analysis, Login Flow, Captcha Handling, Element IDs, Acceptance
Criteria, Validation Rules)
2. How you structured and directed your AI prompts
(Research prompts, generation prompts, code prompts, refinement prompts)
IMPORTANT GUIDELINES
Do not change the provided workspace structure at any cost. You may implement your
framework on top of the existing workspace, but the structure must remain unchanged.
Execution Method: Use the integrated terminal in VS Code for all test execution.
Avoid redundant files and folders and keep the workspace clean and organized.
Create a single defect report named defect_report.json consolidating all identified
defects. This file must be present in your workspace after execution.
Follow industry-standard best practices for framework design, naming conventions,
Search
Esc

Time left
01 : 58
hrs min

Submit

1

and code organization.

Important: Please press **Ctrl + S** frequently to save all your files and avoid losing any changes, and Click the **Tekstac - Evaluate (T)** icon in the left sidebar, then click **Evaluate** to view the evaluation feedback.

APPLICATION UNDER TEST

System Name: SmartUniversity (Rocky Global University)
URL: http://webapps.tekstac.com/SeleniumApp1/SmartUniversit
Login: Username: admin | Password: admin#123

Login notes:
The login page displays a dynamic arithmetic captcha (for example "98 + 65 =") and the computed answer is shown on the page in a result box. Your automation must read the displayed answer and enter it into the captcha field. On valid login a browser alert "Login Successful" is shown — your automation must accept/dismiss the alert (click OK) before the home page (index.html) loads.

Navigation after login:
Students menu → Add Students (add_stud.html)

Search

Time left
01 : 57
hrs min
Submit

1

Professor menu → Add Professor (add_prof.html)

SCOPE

Module 1 — Login: login.html, US-01 to US-05
Module 2 — Add Student: add_stud.html, US-06 to US-13
Module 3 — Add Professor: add_prof.html, US-14 to US-21

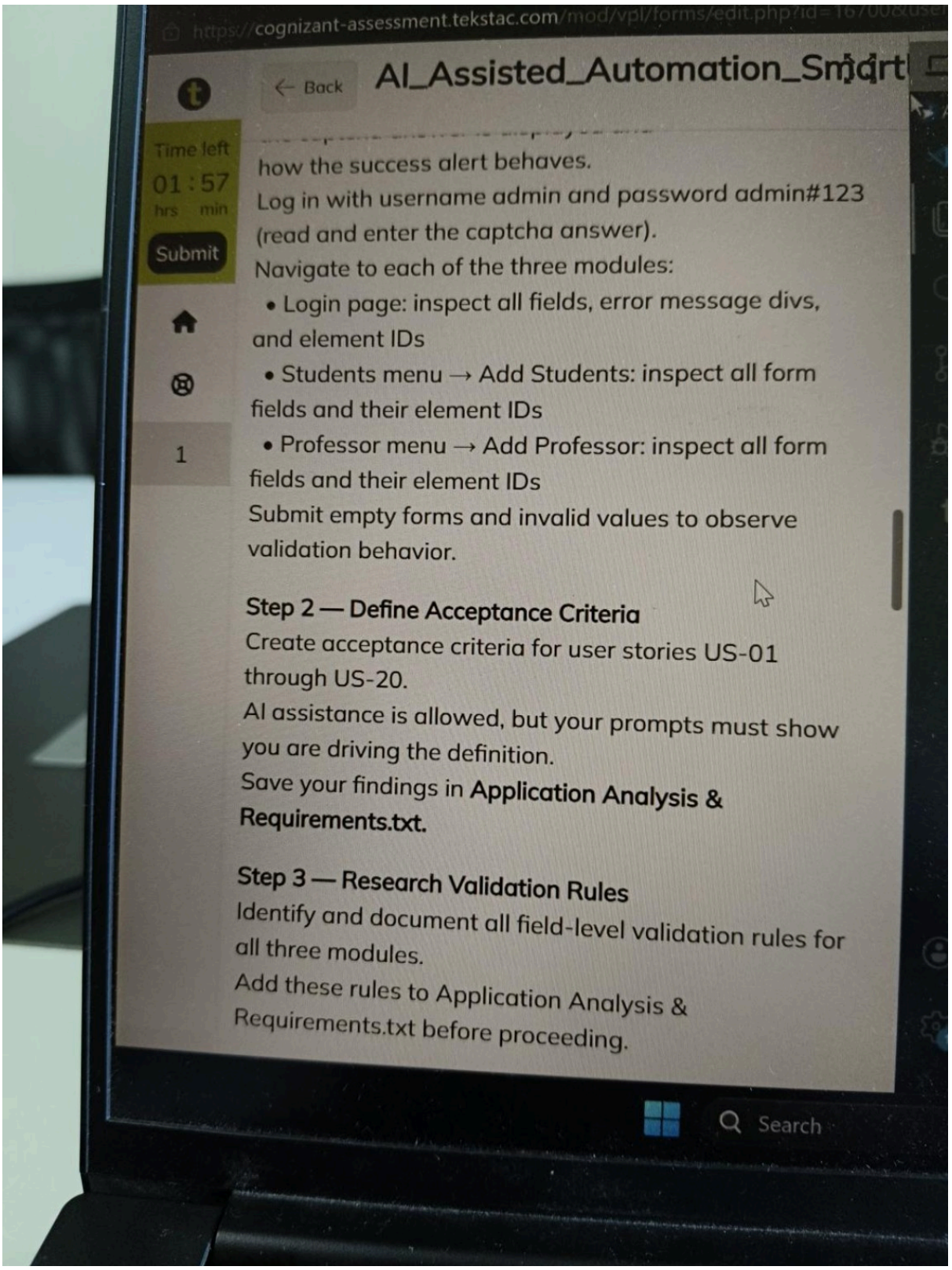
Note: Please refer to the use cases provided in the user story, as outlined in the project template.

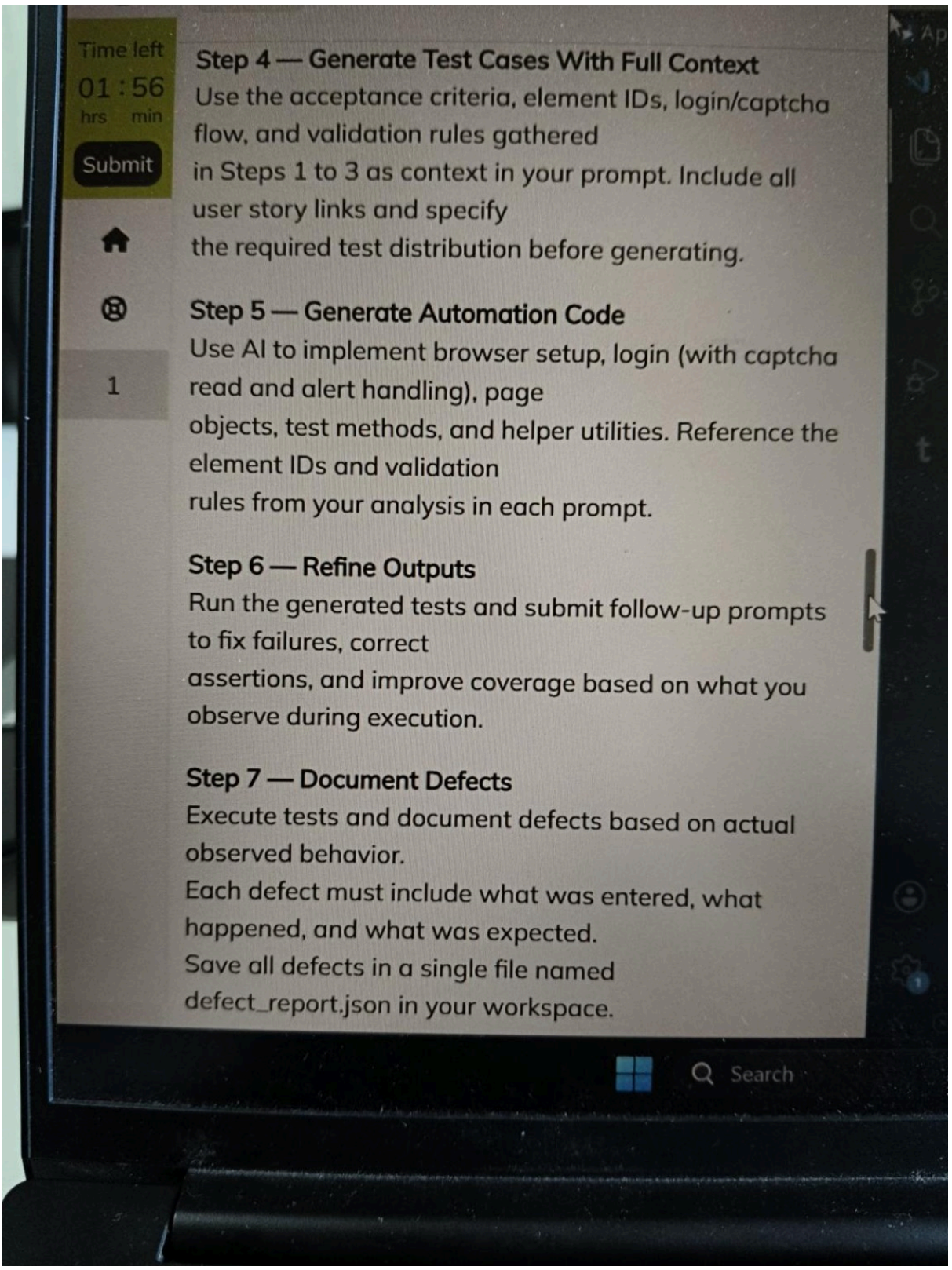
HOW TO APPROACH THIS EXERCISE

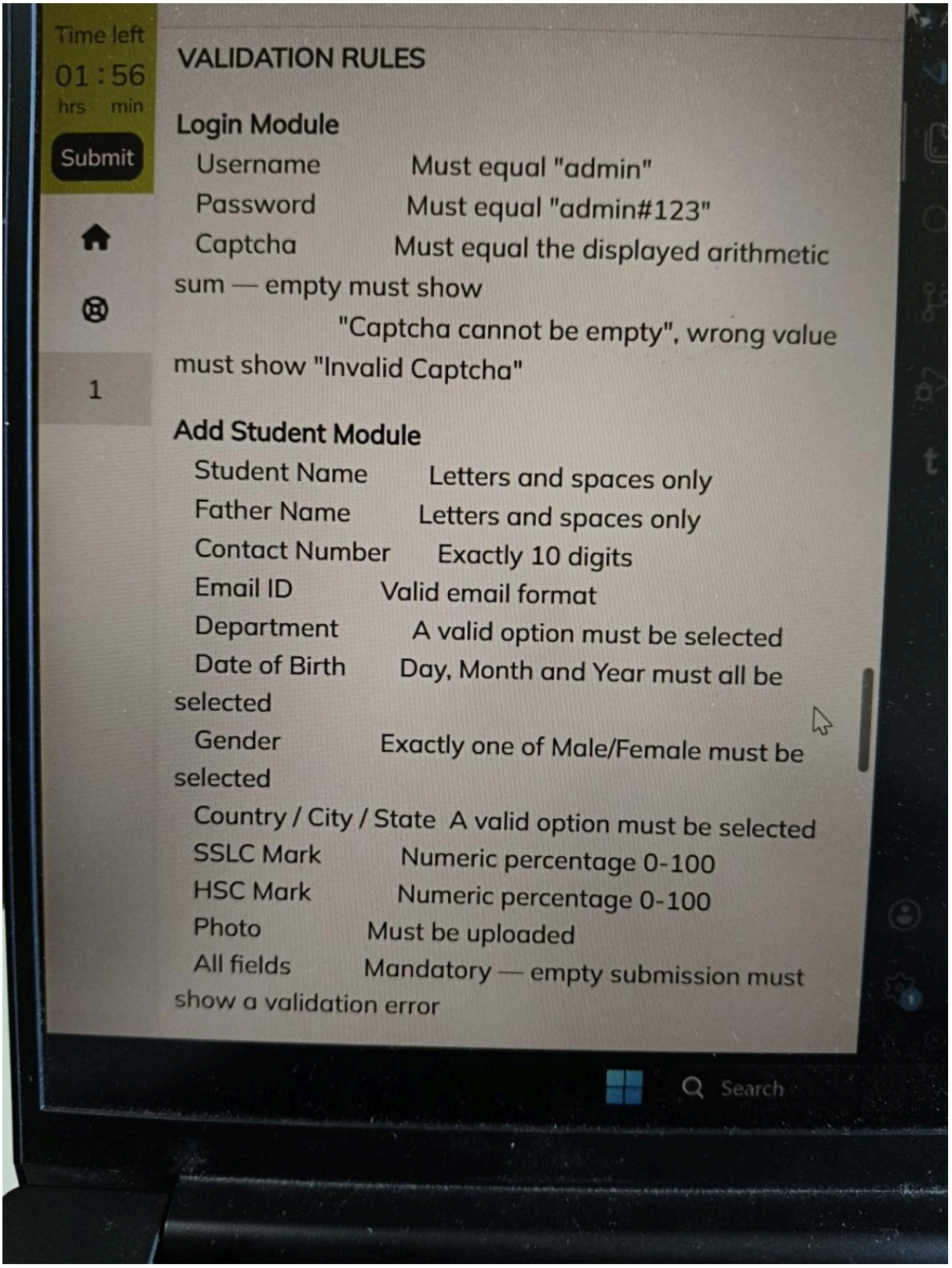
Each step below corresponds to criteria that are scored in your evaluation.
Skipping a step means scoring 0 on the corresponding criteria.

Step 1 — Login and Inspect All Three Modules
Open http://webapps.tekstac.com/SeleniumApp1/SmartUniversit in a browser.
Inspect the login form: username, password, captcha operation, captcha result, captcha input, remember-me, and the submit button. Note how the captcha answer is displayed and

Search


https://cognizant-assessment.tekstac.com/mod/vpl/forms/edit.php?id=
Back
AI_Assisted_Automation_Smart
Time left
01 : 57
hrs min
Submit
1
how the success alert behaves.
Log in with username admin and password admin#123 (read and enter the captcha answer).
Navigate to each of the three modules:
• Login page: inspect all fields, error message divs, and element IDs
• Students menu → Add Students: inspect all form fields and their element IDs
• Professor menu → Add Professor: inspect all form fields and their element IDs
Submit empty forms and invalid values to observe validation behavior.
Step 2 — Define Acceptance Criteria
Create acceptance criteria for user stories US-01 through US-20.
AI assistance is allowed, but your prompts must show you are driving the definition.
Save your findings in Application Analysis & Requirements.txt.
Step 3 — Research Validation Rules
Identify and document all field-level validation rules for all three modules.
Add these rules to Application Analysis & Requirements.txt before proceeding.
Search


Time left
01 : 56
hrs min
Submit
1
Step 4 — Generate Test Cases With Full Context
Use the acceptance criteria, element IDs, login/captcha flow, and validation rules gathered in Steps 1 to 3 as context in your prompt. Include all user story links and specify the required test distribution before generating.
Step 5 — Generate Automation Code
Use AI to implement browser setup, login (with captcha read and alert handling), page objects, test methods, and helper utilities. Reference the element IDs and validation rules from your analysis in each prompt.
Step 6 — Refine Outputs
Run the generated tests and submit follow-up prompts to fix failures, correct assertions, and improve coverage based on what you observe during execution.
Step 7 — Document Defects
Execute tests and document defects based on actual observed behavior.
Each defect must include what was entered, what happened, and what was expected.
Save all defects in a single file named defect_report.json in your workspace.
Search


Time left
01 : 56
hrs min
Submit
1
VALIDATION RULES
Login Module
Username Must equal "admin"
Password Must equal "admin#123"
Captcha Must equal the displayed arithmetic sum — empty must show "Captcha cannot be empty", wrong value must show "Invalid Captcha"
Add Student Module
Student Name Letters and spaces only
Father Name Letters and spaces only
Contact Number Exactly 10 digits
Email ID Valid email format
Department A valid option must be selected
Date of Birth Day, Month and Year must all be selected
Gender Exactly one of Male/Female must be selected
Country / City / State A valid option must be selected
SSLC Mark Numeric percentage 0-100
HSC Mark Numeric percentage 0-100
Photo Must be uploaded
All fields Mandatory — empty submission must show a validation error
Search

Time left
01 : 55
hrs min
Submit

1

Add Professor Module

First Name	Letters and spaces only
Last Name	Letters and spaces only
Phone Number	Exactly 10 digits
Email ID	Valid email format
Department	A valid option must be selected
Date of Birth	A valid date must be selected
Gender	Exactly one of Male/Female must be selected
Qualification	At least one of M.E / Ph.D must be selected — mandatory
Country / City / State	A valid option must be selected
Photo	Must be dragged into the drop box
All fields	Mandatory — empty submission must show a validation error

TASKS

Task 1 — Test Case Generation

Generate at least 60 detailed test cases in CSV format covering all 20 user
stories across all 3 modules.

Required Columns:

TC_ID, Module, User_Story_ID, Test_Scenario,
Test_Type, Priority, Precondition,
Test_Steps, Test_Data, Expected_Result, Actual_Result,

Search

Time left
01 : 55
hrs min
Submit

1

Status, Priority Values: Critical, High, Medium, Low

Distribution: 60-70% of cases must be negative/validation tests (Critical or High priority). Positive/success scenarios should be Low priority. Boundary tests should be Medium.

Task 2 — Automation Implementation

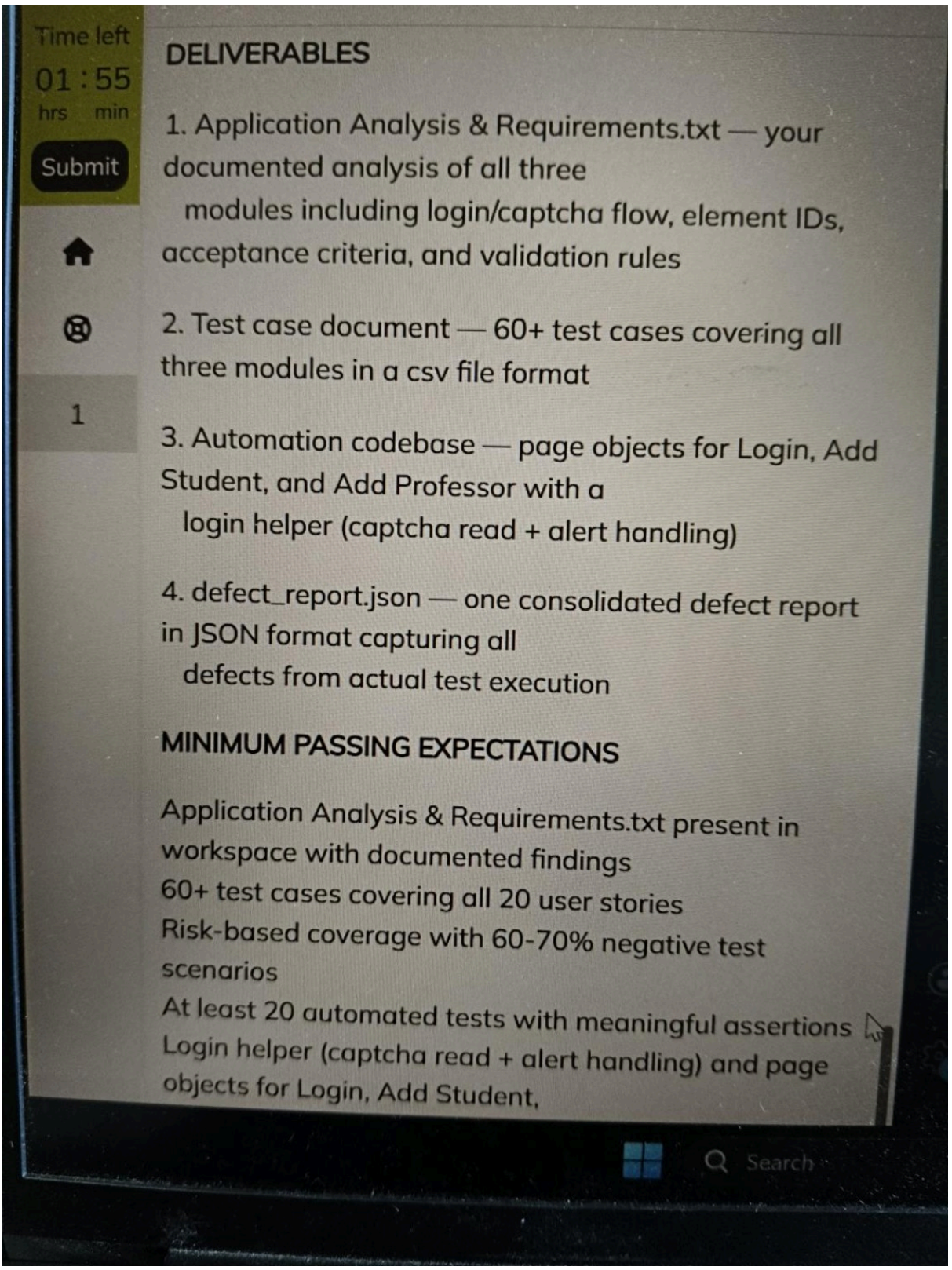
Create a complete automation solution covering all three modules.
Implement at least 20 automated test methods with assertions.
Prioritize critical and high-risk validations over happy-path scenarios.
Every module requires login with admin / admin#123 (with captcha read and success-alert handling) before navigating to the form.

Task 3 — Execution and Bug Detection

Execute automation. Provide execution evidence. Save all defects in a single consolidated file named defect_report.json. Detect and document at least 8 validation defects across all modules. Defects must be from actual test execution — not AI-predicted.

DELIVERABLES

Search



"and add a professor. Defect_report.json present in workspace with at least 8 documented defects, each including expected result, actual result, severity, and priority"

Smart University Answer

ROLE

Act as a senior QA automation engineer, SDET, test architect, requirements analyst, and debugging specialist working directly inside the current VS Code workspace.

You are completing an AI-assisted UI automation assessment for the **SmartUniversity / Rocky Global University** web application.

Your implementation technology is strictly:

- **Playwright**
- **TypeScript**
- Page Object Model
- Reusable fixtures/helpers/utilities
- Playwright assertions
- Integrated VS Code terminal for execution

Your objective is NOT merely to generate code.

Your objective is to:

inspect → analyze → document → derive requirements → identify validation rules → generate comprehensive test cases → implement automation → execute tests → investigate failures → fix automation issues → re-run → identify genuine application defects → document actual defects → perform final regression verification.

The final workspace must contain a complete, executable, maintainable solution satisfying every assessment requirement.

You are assisting me with the **SmartUniversity UI Automation assignment**. Follow the assignment exactly and work with the **existing workspace**.

1. Application Under Test

System: SmartUniversity (Rocky Global University)
URL: `http://webapps.tekstac.com/SeleniumApp1/SmartUniversity`
Login: Username: `admin` | Password: `admin#123`

There are 3 modules:

1. **Login** — `login.html`, US-01 to US-05
2. **Add Student** — `add_stud.html`, US-06 to US-13
3. **Add Professor** — `add_prof.html`, US-14 to US-21

Every module requires login using `admin/admin#123`.

2. Login/Captcha Flow

The login page contains username, password, dynamic arithmetic captcha, captcha result, captcha input, remember-me and submit.

The captcha operation is dynamic, e.g. `98 + 65 =`. The **computed answer is displayed on the page in a result box**. Automation must read that displayed answer and enter it into the captcha field.

For valid login, a browser alert **"Login Successful"** appears. Automation must accept/dismiss the alert before the home page (`index.html`) loads.

After login:

- Students menu → Add Students → `add_stud.html`
- Professor menu → Add Professor → `add_prof.html`

Inspect all fields, element IDs, error-message elements, captcha behavior and validation behavior before implementing tests.

3. Validation Rules

Login

- Username must equal `admin`
- Password must equal `admin#123`
- Captcha must equal displayed arithmetic answer
- Empty captcha → `Captcha cannot be empty`
- Wrong captcha → `Invalid Captcha`

Add Student

- Student Name: letters and spaces only
- Father Name: letters and spaces only
- Contact Number: exactly 10 digits
- Email ID: valid email format
- Department: valid option required
- Date of Birth: Day, Month and Year required
- Gender: exactly one of Male/Female
- Country/City/State: valid option required
- SSLC Mark: numeric percentage 0–100
- HSC Mark: numeric percentage 0–100
- Photo: must be uploaded
- All fields mandatory; empty submission must produce validation errors

Add Professor

- First Name: letters and spaces only
- Last Name: letters and spaces only
- Phone Number: exactly 10 digits
- Email ID: valid email format
- Department: valid option required
- Date of Birth: valid date required
- Gender: exactly one of Male/Female
- Qualification: at least one of M.E / Ph.D mandatory
- Country/City/State: valid option required
- Photo: must be dragged into the drop box
- All fields mandatory; empty submission must produce validation errors

---

4. Required Approach

Follow these assignment steps:

Step 1 — Login and Inspect All Three Modules

Open the application, login with `admin/admin#123`, read the dynamic captcha answer from the page, enter it, handle the success alert, and inspect:

- Login fields, error messages and element IDs
- Add Student fields and element IDs
- Add Professor fields and element IDs
- Empty-form behavior
- Invalid-value behavior
- Captcha behavior and success alert

Step 2 — Acceptance Criteria

Create acceptance criteria for **US-01 through US-20** based on the actual application, fields, validation rules and expected behavior.

Save findings in:

`Application Analysis & Requirements.txt`

Step 3 — Validation Rules

Document all field-level validation rules for all three modules in:

`Application Analysis & Requirements.txt`

Step 4 — Test Case Generation

Generate **at least 60 detailed test cases** covering **all 20 user stories across all 3 modules**.

CSV required columns:

```
TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status
```

Priority values:

```
Critical, High, Medium, Low
```

Required distribution:

- **60–70%** must be negative/validation tests
- Negative/validation tests should be **Critical or High**
- Positive/success tests should be **Low**
- Boundary tests should be **Medium**

Step 5 — Automation Code

Create a complete automation solution for all 3 modules.

Requirements:

- At least **20 automated test methods**
- Every automated test must contain meaningful assertions
- Prioritize critical/high-risk validation scenarios
- Use Page Object Model for Login, Add Student and Add Professor
- Create a reusable login helper
- Login helper must handle dynamic captcha reading and success-alert handling
- Every module must perform login before navigation
- Reference the actual inspected element IDs and validation rules
- Use the existing workspace structure; **do not change it**

You may use Cypress, Playwright or Selenium in JavaScript, TypeScript, Python, Java or C#/.NET, as appropriate for the existing workspace.

Step 6 — Execute and Refine

Execute the automation using the **integrated VS Code terminal**.

Do not simply assume tests pass. Use actual execution results to:

- Fix failures
- Correct assertions
- Improve selectors
- Improve coverage

- Refine the automation based on observed application behavior

Save files frequently using **Ctrl+S**.

Step 7 — Defect Detection

Defects must come from **actual test execution**, not AI predictions.

Document at least **8 validation defects across the modules** in one file:

`defect_report.json`

Each defect must include:

- What was entered/test data
- What actually happened
- Expected result
- Severity
- Priority

---

5. Required Deliverables

The final workspace must contain:

1. `Application Analysis & Requirements.txt`
    - Application analysis
    - Login/captcha flow
    - Element IDs
    - Acceptance criteria for US-01 through US-20
    - Validation rules for all 3 modules
2. **Test case CSV**
    - At least 60 test cases
    - All 20 user stories covered
    - All 3 modules covered
    - Required columns included
    - 60–70% negative/validation coverage
3. **Automation codebase**
    - Page objects for Login, Add Student and Add Professor
    - Login helper with captcha + alert handling
    - At least 20 automated test methods
    - Meaningful assertions
4. `defect_report.json`
    - At least 8 defects
    - Across all modules
    - Based only on actual execution
    - Expected result, actual result, severity and priority included

Important Constraints

- **Do NOT change the provided workspace structure.**
- Do not create unnecessary/redundant files or folders.
- Keep the workspace clean and organized.
- Use the **VS Code integrated terminal** for test execution.
- Do not fabricate element IDs, validation behavior, test results or defects.
- Inspect the application first and base implementation on actual behavior.
- Use AI assistance, but ensure the analysis, requirements and prompts demonstrate that I am directing the process.
- The final solution must satisfy the assignment's evaluation criteria, not merely produce sample code.

Start by inspecting the existing workspace and application. Do not immediately generate code. First determine the existing framework, files, structure, selectors/element IDs and actual application behavior, then proceed through the assignment steps in order.

College Social Network Answer

ROLE

Act as a senior QA automation engineer, SDET, test architect, requirements analyst, and debugging specialist working directly inside the current VS Code workspace.

You are completing an AI-assisted UI automation assessment for the **College Social Network** web application.

Your implementation technology is strictly:

- **Playwright**
- **TypeScript**
- Page Object Model
- Reusable fixtures/helpers/utilities
- Playwright assertions
- Integrated VS Code terminal for execution

Your objective is NOT merely to generate code.

Your objective is to:

inspect → analyze → document → derive requirements → identify validation rules → generate comprehensive test cases → implement automation → execute tests → investigate failures → fix automation issues → re-run → identify genuine application defects → document actual defects → perform final regression verification.

The final workspace must contain a complete, executable, maintainable solution satisfying every assessment requirement.

You are assisting me with the **College Social Network assignment**. Follow the assignment exactly and work with the **existing workspace**.

Analyse the existing project workspace first. Do NOT change, delete, rename, or restructure the provided workspace. Implement the solution on top of the existing structure only. Avoid redundant files/folders.

Use Playwright with TypeScript and Page Object Model. Use the integrated VS Code terminal for installation, execution, debugging, and validation.

The assessment evaluates both:
1. What was analysed/discovered and shared with AI: application analysis, login flow, element IDs, field types, acceptance criteria, and validation rules.
2. How AI prompts were structured: research, generation, code-generation, and refinement/follow-up prompts.

Perform the following steps in order.

APPLICATION UNDER TEST
System: College Social Network
URL: http://webapps.tekstac.com:2121/
Username: admin
Password: admin123

Navigation after login:
Student menu → Register
Student menu → Update Backlog
Faculty menu → Add Faculty

SCOPE
Module 1 — Student Registration: US-01 to US-08
Module 2 — Update Backlog: US-09 to US-11
Module 3 — Add Faculty: US-12 to US-16

Read the user stories/use cases already available in the workspace and treat them as the source of truth.

STEP 1 — LOGIN AND INSPECT ALL THREE MODULES
Login with admin/admin123 and inspect every form.

Identify and document:
- HTML element IDs
- Labels and field types
- Buttons/navigation
- Default values
- Success/error messages
- Form behaviour
- Mandatory fields
- Client/application validation
- Empty-form behaviour
- Invalid-value behaviour

Actually submit empty and invalid data to observe the application. Do not assume behaviour.

STEP 2 — DEFINE ACCEPTANCE CRITERIA
Create acceptance criteria for every user story US-01 through US-16 using the existing user stories plus actual application behaviour.

Create:

Application Analysis & Requirements.txt

It MUST contain:
- Application overview

- Login credentials/flow
- Navigation flow
- Module analysis
- US-01 through US-16
- Acceptance criteria
- Element IDs
- Field types
- Actual observed behaviour
- Validation rules
- Success/error messages
- Any discrepancies discovered

Use this file as context for subsequent AI test-case and code generation.

STEP 3 — RESEARCH VALIDATION RULES
Document all field-level validation rules for all three modules in Application Analysis & Requirements.txt.

Required rules:

Student Registration:
- Student Name: letters and spaces only
- Mobile Number: exactly 10 digits
- Email ID: valid email format
- CGPA: 0.0 to 10.0
- Department Name: non-empty text
- Backlog Count: non-negative integer
- All 6 fields mandatory

Update Backlog:
- Roll Number: valid roll number from a completed student registration
- Backlog Count: non-negative integer
- Both fields mandatory

Add Faculty:
- Faculty Name: letters and spaces only
- Email ID: valid email format
- Designation: non-empty text
- All 3 fields mandatory

Also document any additional rules discovered during actual inspection.

STEP 4 — GENERATE COMPREHENSIVE TEST CASES
Generate AT LEAST 60 detailed test cases covering ALL 16 user stories across all 3 modules.

Create a CSV containing exactly:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Requirements:
- Every US-01 through US-16 must be covered.
- Include positive, negative, validation, boundary, mandatory-field, invalid-format and functional scenarios.
- 60–70% MUST be negative/validation tests.
- Negative/validation tests should be Critical or High priority.
- Positive/success tests should be Low priority.
- Boundary tests should be Medium priority.
- Allowed priorities: Critical, High, Medium, Low.
- Use actual validation rules, element IDs and observed behaviour.
- Do not invent behaviour.
- Update Actual_Result and Status after execution where applicable.

STEP 5 — AUTOMATION IMPLEMENTATION
Create a complete Playwright TypeScript automation solution using Page Object Model.

Required:
- Browser setup/configuration
- Reusable login helper/page
- Student Registration page object
- Update Backlog page object
- Add Faculty page object
- Test methods
- Reusable utilities where needed
- Meaningful assertions
- Reliable Playwright locators
- Proper waits/synchronization

Implement AT LEAST 20 automated test methods covering all 3 modules.

Prioritize:
1. Critical validations
2. High-risk negative tests
3. Boundary tests
4. Positive scenarios

Every module must login using:
admin / admin123

Use the actual element IDs and validation rules discovered during inspection. Do not guess selectors.

STEP 6 — EXECUTE AND REFINE
Run the Playwright tests using the integrated VS Code terminal.

Do not stop at the first execution.

For every failure:
1. Inspect the actual error.
2. Determine whether it is a locator, navigation, timing, test-data, assertion, application-behaviour, or genuine application-defect issue.
3. Fix automation issues.
4. Correct assertions only when actual application behaviour proves the original assertion was wrong.
5. Re-run failed tests.
6. Re-run the complete suite where practical.
7. Improve stability and coverage.

Do NOT remove assertions or weaken valid validations just to make tests pass.

Provide execution evidence through Playwright reports/artifacts where supported.

STEP 7 — DOCUMENT DEFECTS
Execute the validation tests across ALL THREE modules and document defects based ONLY on actual observed execution.

Create exactly ONE consolidated file:

defect_report.json

It must contain AT LEAST 8 documented validation defects.

Each defect must include at minimum:
- Defect ID
- Module
- User Story ID
- Test Case ID
- Test Data / What was entered
- Steps/scenario
- Expected Result
- Actual Result
- Severity
- Priority
- Status

Defects MUST come from actual test execution, not AI prediction or assumptions.

Attempt to identify defects across all three modules. Execute enough negative/boundary/validation tests to obtain genuine observations. Do not fabricate defects merely to reach eight.

FINAL DELIVERABLES
The workspace must contain:

1. Application Analysis & Requirements.txt
   - Analysis of all 3 modules
   - Login flow
   - Element IDs
   - Field types
   - US-01 to US-16
   - Acceptance criteria
   - Validation rules
   - Actual observed behaviour

2. Test case CSV
   - 60+ detailed cases
   - All 16 user stories
   - Required columns
   - 60–70% negative/validation coverage
   - Correct priorities

3. Playwright TypeScript automation codebase
   - POM
   - Login helper
   - Student Registration page
   - Update Backlog page
   - Add Faculty page
   - At least 20 automated tests
   - Meaningful assertions

4. defect_report.json
   - One consolidated JSON file
   - At least 8 actual validation defects
   - Expected result
   - Actual result
   - Severity
   - Priority
   - Execution details

MINIMUM PASSING EXPECTATIONS
Before finishing, verify every item:

- Application Analysis & Requirements.txt exists with documented findings.
- 60+ test cases exist covering all 16 user stories.
- Test coverage is risk-based with 60–70% negative/validation scenarios.
- At least 20 automated tests exist with meaningful assertions.
- Login helper exists.
- Page objects exist for Student Registration, Update Backlog, and Add Faculty.
- All three modules are automated.
- Tests use admin/admin123 login.
- Tests have actually been executed.

- Failed tests have been investigated and refined.
- defect_report.json exists.
- defect_report.json contains at least 8 documented defects.
- Defects are based on actual test execution.
- Every defect includes expected result, actual result, severity, and priority.
- Existing workspace structure remains unchanged.
- No unnecessary/redundant files or folders are created.
- All files are saved and ready for evaluation.

IMPORTANT:
Do not merely explain what should be done. Perform the analysis, inspect the application, create the required files, generate the test cases, implement the Playwright TypeScript framework, execute the tests, debug/refine failures, document actual defects, and leave the completed solution in the workspace.